CODESPRINT 1.0 / DAY 04 / INSTRUCTOR COPY

Greedy Algorithms: C++ Solutions

Ahmmad Nur Swapnil | Lecturer, Department of CSE, Presidency University

11 standalone programs: activity selection, 8 classroom exercises, 1 special challenge, and 1 independent exercise.

Simple C++ only. Plain arrays and `struct`, with `sort()` where needed. No `vector`, `pair`, `string`, or other STL containers.

EXAMPLE 1 / ACTIVITY SELECTION

101081I - Polish Solidarity

O(n log n) | Open Codeforces problem

Greedy idea. Sort meetings by finish time. Attend the next meeting if it starts at or after the previous meeting ends.

Why it works. Choosing the earliest finishing meeting leaves the most time for later meetings.

```cpp
#include <cstdio>
#include <algorithm>
using namespace std;

struct Meeting {
    int start, finish, id;
};

Meeting a[100005];
int chosen[100005];

bool byFinish(Meeting x, Meeting y) {
    if (x.finish != y.finish)
        return x.finish < y.finish;
    return x.start < y.start;
}

int main() {
    int n;
    while (scanf("%d", &n) == 1) {
        for (int i = 0; i < n; i++) {
            int sh, sm, eh, em;
            scanf("%d:%d %d:%d", &sh, &sm, &eh, &em);
            a[i].start = sh * 60 + sm;
            a[i].finish = eh * 60 + em;
            a[i].id = i + 1;
        }

        sort(a, a + n, byFinish);
        int count = 0, lastFinish = -1;
        for (int i = 0; i < n; i++) {
            if (a[i].start >= lastFinish) {
                chosen[count++] = a[i].id;
                lastFinish = a[i].finish;
            }
        }

        for (int i = 0; i < count; i++) {
            if (i > 0) printf(" ");
            printf("%d", chosen[i]);
        }
        printf("\n");
    }
    return 0;
}
```

CLASSROOM / RESOURCE ALLOCATION

1675A - Food for Animals

O(1) | Open Codeforces problem

Greedy idea. Use dog-only and cat-only food first; cover the shortages with universal food.

Why it works. Using restricted supplies first preserves the flexible supply for either animal.

```cpp
#include <iostream>
using namespace std;

int main() {
    int t;
    cin >> t;
    while (t--) {
        long long a, b, c, x, y;
        cin >> a >> b >> c >> x >> y;

        long long needDogs = x - a;
        long long needCats = y - b;
        if (needDogs < 0) needDogs = 0;
        if (needCats < 0) needCats = 0;

        if (needDogs + needCats <= c) cout << "YES\n";
        else cout << "NO\n";
    }
    return 0;
}
```

CLASSROOM / SORT AND PAIR EXTREMES

1843A - Sasha and Array Coloring

O(n log n) | Open Codeforces problem

Greedy idea. Sort the array. Pair the largest remaining value with the smallest remaining value.

Why it works. Moving a larger number toward the positive side and a smaller one toward the negative side never hurts.

```cpp
#include <iostream>
#include <algorithm>
using namespace std;

int a[300005];

int main() {
    int t;
    cin >> t;
    while (t--) {
        int n;
        cin >> n;
        for (int i = 0; i < n; i++) cin >> a[i];
        sort(a, a + n);

        long long answer = 0;
        for (int i = 0; i < n / 2; i++) {
            answer += a[n - 1 - i] - a[i];
        }
        cout << answer << "\n";
    }
    return 0;
}
```

CLASSROOM / EXCHANGE GREEDY

1353B - Two Arrays And Swaps

O(n log n) | Open Codeforces problem

Greedy idea. Sort A and B. Replace small values of A with large values of B while the swap is useful.

Why it works. A useful swap should replace the smallest available A with the largest available B.

```cpp
#include <iostream>
#include <algorithm>
using namespace std;

int a[300005], b[300005];

int main() {
    int t;
    cin >> t;
    while (t--) {
        int n, k;
        cin >> n >> k;
        for (int i = 0; i < n; i++) cin >> a[i];
        for (int i = 0; i < n; i++) cin >> b[i];

        sort(a, a + n);
        sort(b, b + n);
        for (int i = 0; i < k; i++) {
            if (a[i] < b[n - 1 - i]) {
                a[i] = b[n - 1 - i];
            }
        }

        long long sum = 0;
        for (int i = 0; i < n; i++) sum += a[i];
        cout << sum << "\n";
    }
    return 0;
}
```

CLASSROOM / ORDERED MATCHING

1833B - Restore the Weather

O(n log n) | Open Codeforces problem

Greedy idea. Sort A with original positions; sort B; assign matching ranks and restore A's original order.

Why it works. Pairing values of the same sorted rank avoids crossing assignments.

```cpp
#include <iostream>
#include <algorithm>
using namespace std;

struct Item {
    int value, index;
};

Item a[300005];
int b[300005], answer[300005];

bool byValue(Item x, Item y) {
    return x.value < y.value;
}

int main() {
    int t;
    cin >> t;
    while (t--) {
        int n, k;
        cin >> n >> k;
        for (int i = 0; i < n; i++) {
            cin >> a[i].value;
            a[i].index = i;
        }
        for (int i = 0; i < n; i++) cin >> b[i];

        sort(a, a + n, byValue);
        sort(b, b + n);
        for (int i = 0; i < n; i++) {
            answer[a[i].index] = b[i];
        }
        for (int i = 0; i < n; i++) {
            if (i > 0) cout << " ";
            cout << answer[i];
        }
        cout << "\n";
    }
    return 0;
}
```

CLASSROOM / GREEDY SEGMENTATION

1843B - Long Long

O(n) | Open Codeforces problem

Greedy idea. Add absolute values and count negative groups. Zeros do not break a negative group.

Why it works. Each group of negative values needs one sign-flip operation. A zero can be included without harming the result.

```cpp
#include <iostream>
using namespace std;

int main() {
    int t;
    cin >> t;
    while (t--) {
        int n;
        cin >> n;
        long long sum = 0;
        int operations = 0;
        bool insideNegative = false;

        for (int i = 0; i < n; i++) {
            long long x;
            cin >> x;
            if (x < 0) sum -= x;
            else sum += x;

            if (x < 0 && !insideNegative) {
                operations++;
                insideNegative = true;
            }
            if (x > 0) insideNegative = false;
        }
        cout << sum << " " << operations << "\n";
    }
    return 0;
}
```

CLASSROOM / MAINTAIN A FEASIBLE RANGE

1704B - Luke is a Foodie

O(n) | Open Codeforces problem

Greedy idea. Track the minimum and maximum of the current segment.

Why it works. One value can serve every element while max - min <= 2*x.

```cpp
#include <iostream>
using namespace std;

long long a[300005];

int main() {
    int t;
    cin >> t;

    while (t--) {
        int n;
        long long x;
        cin >> n >> x;

        for (int i = 0; i < n; i++) {
            cin >> a[i];
        }

        long long mn = a[0];
        long long mx = a[0];
        int changes = 0;

        for (int i = 1; i < n; i++) {
            if (a[i] < mn) mn = a[i];
            if (a[i] > mx) mx = a[i];

            if (mx - mn > 2 * x) {
                changes++;
                mn = a[i];
                mx = a[i];
            }
        }

        cout << changes << "\n";
    }
    return 0;
}
```

CLASSROOM / RIGHT-TO-LEFT GREEDY

1675B - Make It Increasing

O(n log A) | Open Codeforces problem

Greedy idea. Process from right to left and halve a value only until it is smaller than the next one.

Why it works. Keeping the right neighbor fixed means each left value needs the fewest possible halvings.

```cpp
#include <iostream>
using namespace std;

long long a[300005];

int main() {
    int t;
    cin >> t;
    while (t--) {
        int n;
        cin >> n;
        for (int i = 0; i < n; i++) cin >> a[i];

        int operations = 0;
        for (int i = n - 2; i >= 0; i--) {
            while (a[i] >= a[i + 1] && a[i] > 0) {
                a[i] /= 2;
                operations++;
            }
        }

        bool possible = true;
        for (int i = 0; i + 1 < n; i++) {
            if (a[i] >= a[i + 1]) possible = false;
        }

        if (possible) cout << operations << "\n";
        else cout << -1 << "\n";
    }
    return 0;
}
```

CLASSROOM / MAXIMUM MARGINAL GAIN

1722D - Line

O(n log n) | Open Codeforces problem

Greedy idea. Compute the improvement of each change, sort improvements, and take the best first.

Why it works. Each flip has an independent gain, so the largest available gains are always best.

```cpp
#include <iostream>
#include <algorithm>
using namespace std;

char s[300005];
int gain[300005];

int main() {
    int t;
    cin >> t;
    while (t--) {
        int n;
        cin >> n >> s;
        long long total = 0;

        for (int i = 0; i < n; i++) {
            int current, flipped;
            if (s[i] == 'L') {
                current = i;
                flipped = n - 1 - i;
            } else {
                current = n - 1 - i;
                flipped = i;
            }
            total += current;
            gain[i] = flipped - current;
            if (gain[i] < 0) gain[i] = 0;
        }

        sort(gain, gain + n);
        for (int i = n - 1; i >= 0; i--) {
            total += gain[i];
            cout << total;
            if (i > 0) cout << " ";
        }
        cout << "\n";
    }
    return 0;
}
```

SPECIAL CHALLENGE / SORT AND TWO POINTERS

1690E - Price Maximization

O(n log n) | Open Codeforces problem

Greedy idea. Count base value a[i]/k, then pair remainders to get as many extra +1 points as possible.

Why it works. If smallest + largest is too small, the smallest cannot form a bonus pair with anything.

```cpp
#include <iostream>
#include <algorithm>
using namespace std;

int rem[300005];

int main() {
    int t;
    cin >> t;
    while (t--) {
        int n, k;
        cin >> n >> k;
        long long answer = 0;

        for (int i = 0; i < n; i++) {
            long long x;
            cin >> x;
            answer += x / k;
            rem[i] = x % k;
        }

        sort(rem, rem + n);
        int left = 0, right = n - 1;
        while (left < right) {
            if (rem[left] + rem[right] >= k) {
                answer++;
                left++;
                right--;
            } else {
                left++;
            }
        }
        cout << answer << "\n";
    }
    return 0;
}
```

INDEPENDENT EXERCISE / CONSTRUCTIVE GREEDY

1714C - Minimum Varied Number

O(1) | Open Codeforces problem

Greedy idea. Choose the largest distinct digits first, then print them in ascending order.

Why it works. Large distinct digits make the desired sum with as few digits as possible. Ascending order minimizes the value.

```cpp
#include <iostream>
using namespace std;

int digits[10];

int main() {
    int t;
    cin >> t;
    while (t--) {
        int s;
        cin >> s;
        int count = 0;

        for (int d = 9; d >= 1; d--) {
            if (s >= d) {
                digits[count] = d;
                count++;
                s -= d;
            }
        }

        for (int i = count - 1; i >= 0; i--) {
            cout << digits[i];
        }
        cout << "\n";
    }
    return 0;
}
```